1 Introduction

This laboratory session is designed to understand and implement PID control for DC motor position management. The experiment uses Arduino along with a motor driver to achieve precise control over the motor's angular position by adjusting PID parameters.

2 Experiments

Experiment 1: Visualizing Hall Effect Encoder Data

This experiment involves connecting the DC motor encoder outputs to an oscilloscope. The goal is to analyze the phase relationship and frequency of the Hall Effect A and B signals to determine the motor's direction and speed.

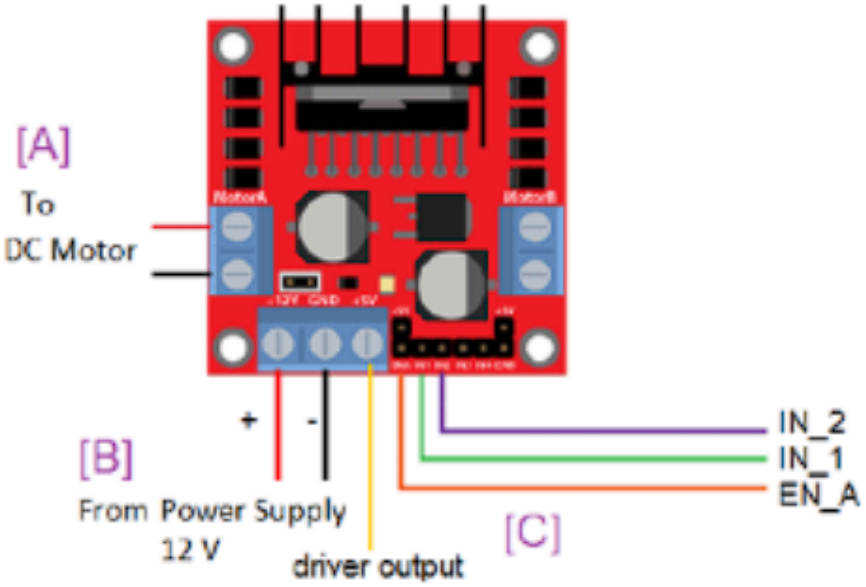


Figure 1: Connection L298N motor driver and Arduino.

Requirements:

- Capture and analyze oscilloscope screenshots for clockwise and counterclockwise motor rotation.
- Determine the phase shift between Hall Effect A and B signals.

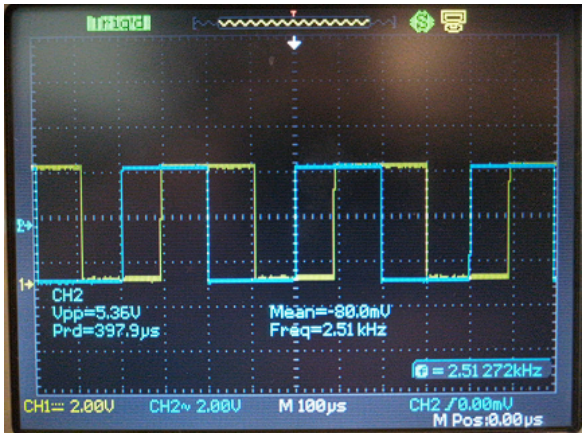


Figure 2: Counter Clock-Wise output.



Figure 3: Clock-Wise output.

Initially, the power wires of the motor were connected. Then, the motor driver was connected to the power supply. The motor driver's ENA pins, which activate the motor, which is connected to the Arduino Mega's two +5V pins. Finally, the IN1 and IN2 pins were separately connected to the motor driver's output pin. We then observed changes in the encoder outputs as the motor's rotation varied.

Experiment 2: DC Motor Position Control Using PID

This experiment introduces a PID controller to regulate the motor's behavior according to set positional requirements. The real-time performance of the controller is observed using the Arduino's serial plotter.

Steps:

1. Implement PID control logic in Arduino to compare the desired position versus actual encoder readings.
2. Adjust and tune K_p, K_i, and K_d to observe different control dynamics.
3. Capture and discuss the serial plotter outputs showing the PID controller's response.

Results:

- Demonstration of PID tuning effects: stability, overshoot, and response time.
- Comparative analysis of P-only, PD, and PID control on motor position stability.

2.2.1 Control with the Proportional Term

The following code was used to control the motor using only the proportional term:

```
define encoderPinA 2 // motor encoder channel A
define encoderPinB 3 // motor encoder channel B
define gearRatio 18.75 // Gearbox Ratio of the actuator
define encoderRes 64 // Encoder Resolution for one rotation
define piNum 3.141
long CurrentTime = 0; // to calculate the time
int enA = 8; // motor driver enable pin (pwm )
int in1 = 9; // motor driver input 1 pin
int in2 = 10; // motor driver input 2 pin
float Err = 0; // the error value
float PreErr = 0;
float DErr = 0;
float IErr = 0;
float PreIErr = 0;
float U = 0; // the result value (to pwm)
float Kp = 10; // constant for the proportional term ..... (X1)
float Ki = 0;
float Kd = 0;
float Reference = 0; // intial value for the reference motor angle
volatile long encoderCount =0; // intial value for the encoder count
long newCount ;
long oldCount = 0;
float pos = 0; // intial value for the actual motor angle
float previousTime = 0;
float interval ;
void setup (){
pinMode ( encoderPinA , INPUT_PULLUP );
pinMode ( encoderPinB , INPUT_PULLUP );
pinMode ( enA , OUTPUT );
pinMode ( in1 , OUTPUT );
pinMode ( in2 , OUTPUT );
attachInterrupt ( digitalPinToInterrupt ( encoderPinA ), HallEffectSensor , RISING );
 // when there is a 'RISING ' edge , make an interrupt and call the 'HallEffectSensor ' function .
attachInterrupt ( digitalPinToInterrupt ( encoderPinB ), HallEffectSensor , RISING );
Serial . begin (9600);
}
void loop (){
previousTime = CurrentTime ;
 CurrentTime = millis (); // Seconds
 interval = CurrentTime - previousTime ;

 // This is a step function , start after 3 sec
 if( CurrentTime >3000){
 Reference = piNum /2; // 90 Degrees ( You can change this )
 }
 else {
 Reference = 0; // [ Degree ]
 }

PreErr = Err ;
```

```
Err = Reference - pos ;
 //
 //
// Write your errors for integral and derivative parts !!!
 //
 //

 // PID control ( edit Ki and Kd to make PI control or PID control )
 U = Kp * Err ; // ADD INTEGRAL AND DERIVATIVE PARTS HERE !!!

 if (U >255){
 U =255; // the pwm can 't be more than 255
 }
 else if(U < -255){
 U = -255;
 }
 else {
 U = U ;
 }

 // run the motor clockwise or counter clockwise
if (U >=0){
 digitalWrite ( in1 , HIGH );
 digitalWrite ( in2 , LOW );
 analogWrite ( enA , U ); // write the pwm value to the motor driver enable pin
 }
 else {
 digitalWrite ( in1 , LOW );
 digitalWrite ( in2 , HIGH );
 analogWrite ( enA , U ); // write the pwm value to the motor driver enable pin
 }


 newCount = encoderCount ;

 // converting the decoder reads to an angle value ( position )
 pos = pos + (( newCount - oldCount ))*(2* piNum /( encoderRes * gearRatio )); // pulses * encoder
 // 0.098 is the rotation ( radians ) per one pulse , 0.0533 is the gear ratio : 1:18.75

 Serial . print ( pos );
 Serial . print ("\t");
 Serial . println ( Reference );

 oldCount = newCount ;
}
 // HallEffectSensor function
 void HallEffectSensor (){
 if ( digitalRead ( encoderPinA ) == LOW && digitalRead ( encoderPinB ) == HIGH ) {
 encoderCount ++; // CW Rotation
 }
 else if ( digitalRead ( encoderPinA ) == HIGH && digitalRead ( encoderPinB ) == LOW ) {
 encoderCount - -; // CCW Rotation
 }
 else {
```

```
encoderCount = encoderCount ;
}
}
```

In this setup, we tested various values for the proportional gain, Kp. The following figures illustrate the motor's response at different Kp values:

- Figure 4: With Kp = 10, the motor did not move.

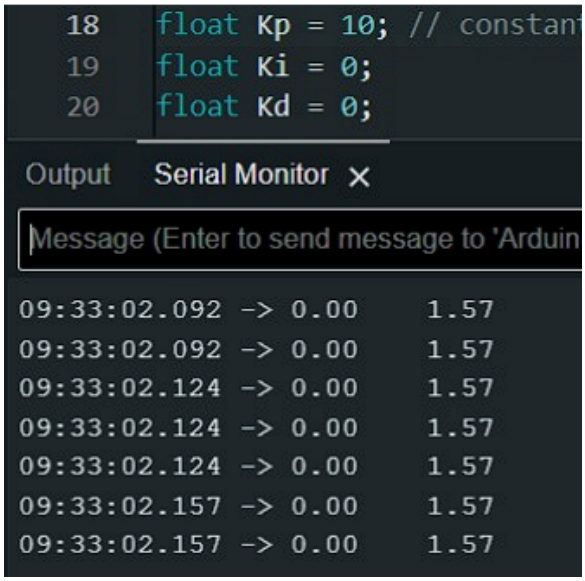


Figure 4: Kp = 10.

- Figure 5: With Kp = 20, the motor moved but significantly undershot the target position.

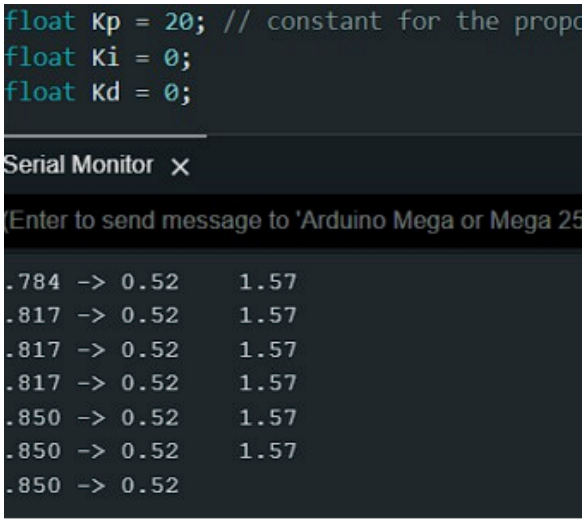


Figure 5: Kp = 20.

- Figure 6: With Kp = 40, the motor's response was closer to the target but involved overshooting.

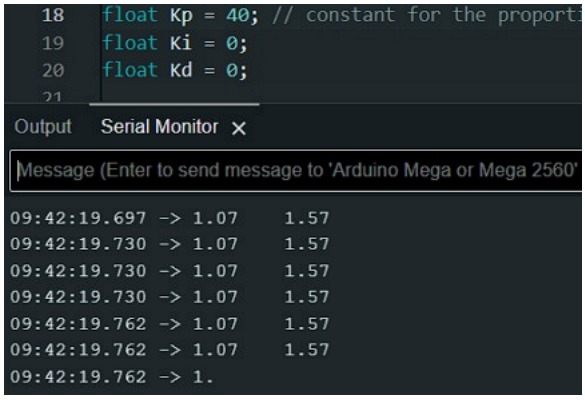


Figure 6: Kp = 40.

2.2.2 Control with the PID

In this part, we modified the Arduino code to incorporate both integral and derivative terms, enhancing the PID control capabilities. These modifications allows for a more complex control strategy, aiming to stabilize the system response. Unfortunatelly, serial plotter was not working properly so we were not able to test the modification and observe the changes.

We have added that part of the code as modification.

```
PreErr = Err ;
Err = Reference - pos ;
//
//
// Write your errors for integral and derivative parts !!!
DErr = ( Err - PreErr )/ interval ;

IErr = (( Err + PreErr )* interval )/2 + PreIErr ;

PreIErr = IErr ;
//
//

// PID control ( edit Ki and Kd to make PI control or PID control )
U = Kp * Err + Kd * DErr + Ki * IErr ; // ADD INTEGRAL AND DERIVATIVE PARTS HERE !!!
```

3 Conclusion

The experiments conducted provided insights into PID control and its practical application to DC motor position management. Adjustments in PID parameters significantly affected the motor's response.